RE:봄 주말 교육 · 1부 · 2026-11-07(토)

LLM과 API 기초

API를 부르는 법부터 토큰 · 비용 · 파라미터까지

실습 노트북 01 API 기초 · 02 토크나이저 · 12 Responses API (코랩)

오리엔테이션

9시간 지도: 기초를 먼저 쌓고, 프로젝트는 질문으로 잇는다

	날짜	주제	다루는 것
1부	11/07	LLM과 API 기초	API · 토큰 · LLM 동작 원리 · 추론 모델 · 비용과 캐싱 · 파라미터
2부	11/14	모델 다루기와 임베딩	오픈 모델 · 양자화 · 로컬 서빙 · 프롬프트 설계 · 임베딩과 유사도 검색
3부	11/21	검색 · RAG · 에이전트	청킹 · RAG · 하이브리드 검색 · 툴 콜링 · 에이전트
W4	11/28	적용 클리닉	새 자료 없음. 각자 담당 코드를 가져와 같이 본다

RE:봄 블록마다 **RE:봄이라면?** 질문이 한 장씩 있다. 답은 화요일 리뷰에서 같이 맞춰 본다

오리엔테이션

오늘의 흐름: 설명하고 바로 실습하기를 세 번 반복한다



시작	블록	설명	실습
0:00	오리엔테이션	지도 · 실습 환경 · 키 규칙	
0:10	A · API 기초	요청과 응답 · 상태 코드 · SDK · FastAPI	01 API 기초
0:50	B · 토큰과 LLM의 동작	토큰 · 컨텍스트 · 다음 토큰 예측 · 환각	02 토크나이저
1:55	C · API를 제대로 쓰기	추론 · 비용 · 캐싱 · 파라미터 · 응답 객체	12 Responses API
2:50	정리	열 문장 요약 · 화요일까지 할 일	

오리엔테이션

실습 환경과 API 키 규칙

실습 환경

- 코랩에서 github.com/acho98/rebom-lec 의 labs/ 노트북을 연다
- 01 · 02 · 12는 CPU 런타임이면 된다
- 데이터는 노트북이 저장소에서 알아서 받는다
- 노트북 맨 위의 코랩 배지를 누르면 바로 열린다

API 키 규칙

- 키는 노트북의 입력창(getpass)에만 붙여 넣는다
- 코드 · 출력 · 깃 · 메신저에 키를 쓰지 않는다
- 수업용 키는 지출 한도가 걸려 있고, 수업 뒤 폐기한다
- 공개 저장소에 올라간 키는 발견 즉시 자동 폐기된다

주의 키가 새면 순서는 **폐기가 먼저**, 이력 정리는 나중이다. 지워도 이미 복제됐을 수 있다

A

API 기초

프로그램끼리는 어떻게 말을 주고받을까?

- 요청과 응답 · 상태 코드
- requests vs SDK · OpenAI 호환 규격
- AI API의 네 가지 종류 · FastAPI로 내 API 만들기

설명 20분 → 실습 01 · 20분

API는 요청과 응답의 약속이다

정해진 주소로 **요청**을 보내면, 정해진 형식으로 **응답**이 돌아온다

요청: 내 코드 → 서버

```
POST /v1/responses
Host: api.openai.com
Authorization: Bearer <키>
Content-Type: application/json

{"model": "gpt-6-luna",
 "input": "안녕?"}
```

응답: 서버 → 내 코드

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "completed",
 "output": [{
   "type": "message",
   "content": [{
     "text": "안녕하세요!"}]}]}
```

핵심 웹페이지도 같은 HTTP다. 다른 점은 받는 쪽이 사람이 아니라 **프로그램**이라는 것

요청은 네 부분: 메서드 · 주소 · 헤더 · 본문

부분	예	뜻
메서드	GET · POST	가져오기 · 보내서 처리하기
주소 (URL)	https://api.openai.com/v1/responses	어느 서버의 어느 기능인가
헤더	Authorization: Bearer 〈키〉	누가 보내나 (인증) · 본문 형식
본문	{"model": "gpt-6-luna", "input": "안녕?"}	무엇을 처리할까 (JSON)

핵심 GET은 조건을 주소 뒤에 붙인다: ?page=1&size=100. 공공데이터 API가 대부분 이 방식이다

상태 코드 첫 자리로 누구 문제인지부터 가른다

2xx · 성공

- 200 OK
- 본문(JSON)을 읽는다

4xx · 내 요청 문제

- 400 형식 · 파라미터 오류
- 401 키가 없거나 틀림
- 403 권한 없음
- 404 주소 · 모델 이름 틀림
- 422 값 검증 실패
- 429 너무 많이 · 한도 초과

5xx · 서버 문제

- 500 · 502 · 503
- 제공자 장애 · 과부하
- 잠시 뒤 다시 보낸다

핵심 4xx는 **고쳐서** 다시 보낸다. 429와 5xx만 **기다렸다가** 다시 보낸다

같은 호출, 두 가지 방법: requests로 직접 vs SDK

requests: HTTP를 직접 만든다

```python
import os, requests
key = os.environ["OPENAI_API_KEY"]
r = requests.post(
    "https://api.openai.com/v1/responses",
    headers={"Authorization": f"Bearer {key}"},
    json={"model": "gpt-6-luna", "input": "hi"},
)
print(r.status_code)          # 200
print(r.json()["output"])     # 답이 깊이 들어 있다
```

OpenAI SDK

```python
from openai import OpenAI
client = OpenAI()  # 키: 환경변수
r = client.responses.create(
    model="gpt-6-luna",
    input="hi",
)
print(r.output_text)
```

핵심 SDK가 대신 해 주는 것: 주소 · 인증 헤더 · JSON 변환 · 타입 · **재시도(기본 2회)**

OpenAI 호환 규격: base_url만 바꾸면 다른 서버도 같은 코드로

```
cloud = OpenAI()                                          # OpenAI
local = OpenAI(base_url="http://localhost:11434/v1",
               api_key="ollama")                          # 내 GPU의 Ollama
clova = OpenAI(base_url="https://clovastudio.stream.ntruss.com/v1/openai",
               api_key=CLOVA_KEY)                         # 네이버 클라우드

# 부르는 코드는 같다. 바뀌는 것은 model 이름뿐
r = local.chat.completions.create(model=LOCAL_MODEL, messages=msgs)
```

핵심 Ollama · vLLM · 여러 클라우드가 같은 규격을 따른다. 모델을 바꿔도 **코드는 거의 그대로**. 2부 실습 05에서 직접 해 본다

AI API는 하는 일에 따라 네 가지로 나뉜다

생성
- 글 · 이미지 → 글
- 답변 · 요약 · 추출 · 분류 (오늘)

임베딩
- 글 · 이미지 → 숫자 벡터
- 의미가 가까운 것 찾기 (2부)

검색
- 질의 → 관련 문서 목록
- 벡터 DB · 키워드 · 웹 (3부)

결정
- 입력 → 정해진 선택지 + 점수
- 라우팅 · 채점 · 안전 판정

핵심 같은 모델이라도 **어떤 API로 부르느냐**에 따라 돌려받는 것이 다르다

내 함수도 API가 된다: FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Req(BaseModel):
    text: str
    max_len: int = 50

@app.post("/summarize")
def summarize(req: Req):
    return {"summary": req.text[:req.max_len]}
```

FastAPI가 알아서 해 주는 것

- /docs 에 문서 화면이 생긴다
- JSON ↔ 파이썬 객체 변환
- 타입이 틀리면 422 + 어느 필드가 틀렸는지 알려 준다
- 실습 01 §4~5에서 직접 422를 받아 본다

배운 것을 우리 프로젝트에 대 보기

RE:봄에는 API가 세 방향으로 나온다: 받아 오는 API(공공데이터 구조동물) · 부르는 API(임베딩 · 생성) · 만드는 API(검색)

Q1 공공데이터 API가 429나 503을 돌려주면, 수집 작업은 어떻게 해야 할까?

Q2 우리가 만들 검색 API는 어떤 입력에 422를 돌려줘야 할까?

Q3 공공데이터 키와 OpenAI 키는 어디에 두고, 코드에서 어떻게 읽을까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

실습 01 · 20분

실습 01: API 기초

코랩에서 열기: labs/01_api_basics.ipynb

순서

- §1 공개 API를 requests로 부르고 JSON 읽기
- §2 OpenAI를 SDK 없이 직접 부르기
- §3 같은 호출을 SDK로
- §4 FastAPI로 내 API 만들기
- §5 잘못된 요청 보내기 (422)
- §6 모델도 API다

보면서 확인할 것

- 상태 코드와 응답 JSON을 직접 출력해 본다
- requests와 SDK 코드 길이의 차이
- 422 응답의 detail에 무엇이 적혀 있나
- 끝나면 🤔 생각해보기 셀에 답을 적는다

B

토큰과 LLM의 동작

모델은 글을 어떻게 읽고, 어떻게 쓸까?

- 토큰 · 한국어 · 숫자
- 입력·출력 토큰 · 컨텍스트 윈도우 · 누적 대화
- 다음 토큰 예측 · temperature · 학습 단계 · 환각

설명 30분 → 실습 02 · 25분

모델은 글자가 아니라 토큰을 읽는다

문자열
"한빛테크 출장 규정"

→

토크나이저
조각으로 나눈다

→

토큰 ID
〔3748, 122164, …〕

→

모델
ID로 계산한다

입력	토큰 조각 (o200k_base)	토큰 수
안녕하세요	'안' · '녕하세요'	2
Hello	'Hello'	1
한빛테크 출장 규정	'한' '빛' '테' '크' ' 출장' ' 규' '정'	7

핵심 자주 나오는 말은 한 토큰, 드문 고유명사는 **글자 단위로** 쪼개진다

B · 토큰

토큰 수가 비용 · 한도 · 속도를 정한다

비용

- 요금 = 토큰 수 × 단가
- 입력 · 출력 단가가 따로

한도

- 한 번에 넣고 받는 양도 토큰으로 센다
- 넘으면 400 오류

속도

- 출력은 한 토큰씩 만든다
- 긴 답 = 긴 대기

핵심 API를 쓸 때는 글자 수가 아니라 **토큰 수**로 생각한다

같은 뜻이라도 한국어는 토큰을 더 쓴다

1.92배
cl100k_base (GPT-4 세대)
영어 대비 한국어 토큰 수

1.27배
o200k_base (GPT-4o 이후)
영어 대비 한국어 토큰 수

+38%
같은 규정집 문서
o200k 2,538 → cl100k 3,507 토큰

토크나이저 사전에 한국어 조각이 많을수록 덜 쪼갠다. 그래서 **모델을 바꾸면 같은 글의 비용도 바뀐다**

실습 실습 02에서 내 문장으로 직접 센다. 토크나이저 네 종류를 나란히 비교한다

숫자와 공백도 조각난다: 계산은 코드에 맡긴다

숫자

- '30,000' → '30' · ',' · '000'
- 모델은 숫자를 값이 아니라 조각으로 본다
- 합계 · 크기 비교를 틀릴 수 있다

공백

- '출장'과 ' 출장'은 다른 토큰 ID
- 앞 공백 하나로 모델이 보는 입력이 바뀐다
- 표기를 통일하면 결과가 안정된다

핵심 숫자 계산 · 크기 비교 · 날짜 계산은 LLM이 아니라 코드로 한다

요청 전체가 입력 토큰, 추론과 답이 출력 토큰이다

입력 토큰
지시 + 대화 기록 + 질문 + 문서 + 도구 정의

→

모델
gpt-6-luna

→

출력 토큰
추론 토큰(안 보임) + 답 토큰

응답의 usage (API 문서의 예시 값)

```
"usage": {
  "input_tokens": 328,
  "input_tokens_details": {"cached_tokens": 0},
  "output_tokens": 52,
  "output_tokens_details": {"reasoning_tokens": 0},
  "total_tokens": 380
}
```

읽는 법

- input: 보낸 것 전부
- cached: 그중 캐시 적중
- output: 추론 + 답
- reasoning: 그중 추론

핵심 청구서는 usage에 있다. 추론 토큰은 **안 보이지만 출력 단가로** 과금된다

컨텍스트 윈도우: 입력과 출력이 함께 쓰는 한 번의 작업 공간

1.05M
GPT-6 Luna
최대 출력 128K

1M
Claude 5.x 계열
최대 출력 128K

8K
Llama 3 8B (2024)
2년 만에 100배 넘게

규칙

- 입력 + 출력(추론 포함)의 합이 한도 안이어야 한다
- 넘으면 400 오류. 알아서 줄여 주지 않는다

크다고 해결되지 않는다

- 길수록 비싸고 느려진다
- 중간에 있는 정보를 놓친다 (Lost in the Middle)
- 필요한 것만 골라 넣는다 → 3부 RAG

B · 누적 대화

모델은 앞 대화를 기억하지 않는다: 매 턴 전부 다시 보낸다

instructions 너는 출장 규정 안내 담당이다

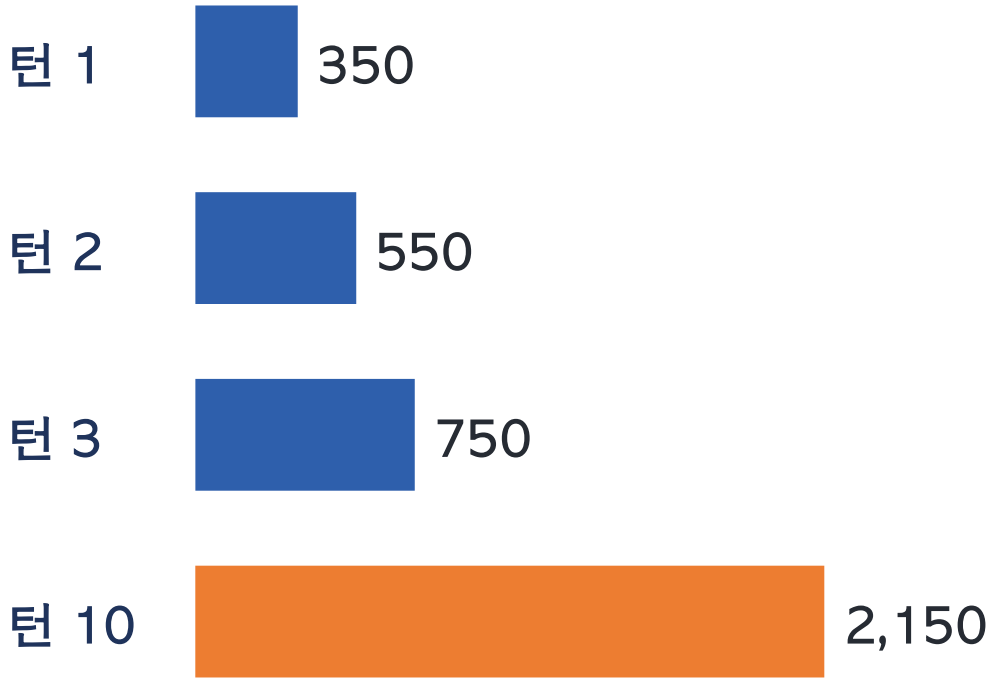
user 과장 일비는 얼마인가요? (턴 1)

assistant 하루 30,000원입니다

user 부장은요? (턴 2)

assistant 하루 40,000원입니다

user 둘 다 3일이면 합계는? (턴 3)



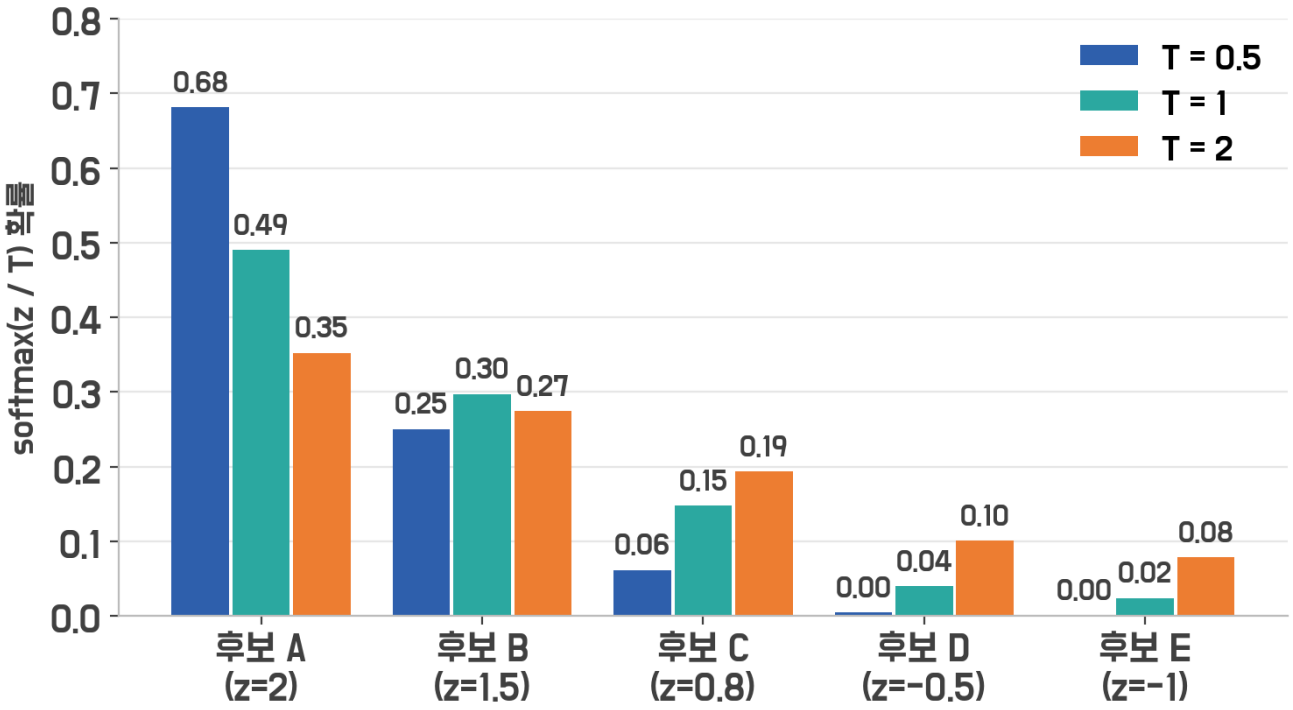
핵심 기억하는 것처럼 보이는 건 **앱이 기록을 다시 보내기** 때문이다. 대화가 길수록 매 턴 입력이 커진다

LLM은 다음 토큰 하나를 고르는 일을 반복한다

지금까지의 글	다음 토큰 후보와 확률 (예시)	고른 토큰
과장 일비는	하루 0.62 · 3 0.21 · 규정 0.08 …	하루
과장 일비는 하루	30 0.71 · 3 0.12 · 최대 0.05 …	30
과장 일비는 하루 30	,000 0.93 · 만 0.04 …	,000
…	끝 토큰이 나올 때까지 반복	(끝)

핵심 답을 한 번에 만드는 게 아니다. 토큰마다 한 번씩 계산하므로 **출력이 길수록 느리다**

temperature: 확률 분포를 뾰족하게 또는 평평하게 만든다



쓰는 법

- 낮게(0~0.3): 매번 비슷한 답 · 추출 · 분류
- 높게(0.8~): 다양한 답 · 아이디어
- 추론 모델은 effort가 none일 때만 받는다
- top_p와 둘 중 하나만 바꾼다

핵심 T가 낮으면 1등 후보가 더 독차지하고, 높으면 낮은 후보에게도 기회가 간다

같은 질문에 다른 답이 나오는 두 가지 이유

① 샘플링

- temperature 〉 0이면 매번 다른 토큰이 뽑힐 수 있다
- 확률이 비슷한 후보가 많을수록 더 흔들린다

② 서버 환경

- 연산 순서 · 배치 · 정밀도의 미세한 차이
- temperature 0이어도 **완전히 같다는 보장은 없다**
- 모델 버전이 바뀌면 분포 자체가 바뀐다

핵심 같은 결과가 필요하면 형식은 스키마로 고정하고, 한 번 얻은 결과는 **저장해서 다시 쓴다**

LLM은 세 단계로 학습한다: 사전학습 → SFT → 선호 정렬

사전학습
인터넷 규모의 글로 다음 토큰 예측 → 언어와 지식

SFT (지도 미세조정)
질문 · 답 예시로 대화 형식 → 지시를 따른다

선호 정렬
RLHF · DPO · GRPO → 사람이 더 좋아하는 답

API에서 보이는 흔적

- instructions · user · assistant 역할 = SFT 때 배운 대화 형식 (채팅 템플릿)
- '모른다'고 하거나 거절하는 태도 = 정렬 단계에서 배운 것
- 지식 컷오프 = 사전학습 데이터가 끝난 시점. 그 뒤 일은 모른다

환각: 근거가 없어도 그럴듯한 답을 만든다

근거 없이 '과장 숙박비 상한은?' → **"1박당 10만 원"** (실제 규정: 110,000원 · 오픈 모델 1.2B 실측)

줄이는 방법	한계
근거 문서를 넣는다 (RAG · 3부)	검색이 틀리면 답도 틀린다
'문서에 없으면 모른다고 답하라' + 출처 요구	지시만으로 완전히 막지 못한다
정확한 숫자는 DB · 코드로 확인한다	확인할 원천 데이터가 있어야 한다

핵심 원인은 학습 목표다. 다음 토큰을 예측할 뿐 **생성 중에 사실을 확인하지 않는다**

모델이 모르는 것을 채우는 세 가지 길

방법	무엇을 바꾸나	맞는 경우	비용
프롬프트	지시 · 예시	형식 · 말투 · 규칙	가장 싸다
RAG	입력에 넣는 자료	최신 · 사내 지식, 출처가 필요할 때	검색 시스템
파인튜닝	모델 가중치	같은 형식 · 판단을 대량으로, 작은 모델 특화	데이터 · 학습

핵심 보통 순서는 프롬프트 → RAG → 파인튜닝. **지식이 부족하면 RAG, 행동을 바꾸려면 파인튜닝**

배운 것을 우리 프로젝트에 대 보기

공고 한 건에는 품종 · 나이 · 체중 · 특징 문구가 있다. 특징 문구에서 태그를 뽑아 검색 필터로 쓸 계획이다

Q1 공고 한 건은 몇 토큰일까? 하루 500~600건, 3년치 약 29만 건이면?

Q2 특징 문구에 '순함'이 없는데 모델이 '온순' 태그를 붙이면, 그 필터로 찾는 사람에게 무슨 일이 생길까?

Q3 나이 · 체중 같은 숫자는 LLM에게 맡길까, 코드로 파싱할까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

실습 02 · 25분

실습 02: 토크나이저

코랩에서 열기: labs/02_tokenizer.ipynb

순서

- §1 토큰은 글자가 아니다
- §2 한국어 토큰 페널티
- §3 토크나이저마다 다른 토큰 수
- §4 숫자 · 공백 · 유니코드
- §5 문서 견적: 토큰 × 단가
- §6 추정 vs 실제 usage

보면서 확인할 것

- 내 문장 하나로 한국어 · 영어 토큰 수 비교
- tiktoken 추정과 API usage의 차이
- 문서 하나를 처리하는 비용은 얼마인가
- 끝나면 🤔 생각해보기 셀에 답을 적는다

휴식 10분

다음: C · API를 제대로 쓰기 (추론 · 비용 · 캐싱 · 파라미터)



API를 제대로 쓰기

얼마가 들고, 무엇을 조절할 수 있을까?

- 추론 모델과 reasoning.effort
- 단가 · 비용 계산 · 프롬프트 캐싱
- 파라미터 · 출력 한도 · 구조화 출력 · 응답 객체 · 로그

설명 30분 → 실습 12 · 25분

추론 모델은 답하기 전에 보이지 않는 생각 토큰을 만든다

질문
입력 토큰

→

추론 토큰
안 보임 · 출력 단가 · 출력 한도에 포함

→

답 토큰
output_text

좋아지는 것

- 조건이 여러 개인 판단 · 다단계 계산
- 코드 작성 · 디버깅
- 도구를 어떤 순서로 쓸지 계획

치르는 것

- 추론 토큰만큼 비용이 든다
- 첫 글자가 나오기까지 오래 걸린다
- 출력 상한이 작으면 생각만 하다 끝난다

핵심 생각의 양은 reasoning.effort로 정한다: none · low · medium(기본) · high …
(모델마다 지원값이 다르다)

C · 추론 모델

effort를 올리면 추론 토큰과 비용이 함께 는다

effort	추론 토큰 (가정)	출력 토큰 합계	1건 비용	1,000건
low	300	500	$0.00051	$0.51
high	3,000	3,200	$0.00186	$1.86
high ÷ low	×10	×6.4	×3.6	×3.6

가정: gpt-6-luna · 입력 2,598토큰(문서 2,538 + 질문 60) · 답 200토큰. 추론 토큰 수는 가정값이고, 실제는 usage의 reasoning_tokens로 잰다

핵심 추론이 10배여도 총비용은 3.6배: 입력과 답 비용은 그대로라서. 그래도 **대량 처리에서는 이 차이가 쌓인다**

단순 추출 · 분류는 끄고, 여러 단계의 판단에만 켠다

끄는 과제 (none)

- 단순 분류 · 추출
- 정해진 형식 변환 (요약 · JSON · 번역)
- 수천 건 대량 반복 · 빨라야 하는 화면

켜는 과제 (low 이상)

- 여러 조건을 조합하는 판정
- 계산이 여러 단계인 문제
- 코드 작성 · 디버깅

정하는 법: 평가셋으로 잰다

- 질문 + 정답 수십 개로 effort별 정확도 · reasoning_tokens · 시간을 기록한다
- 기준을 넘는 설정 중 **가장 싼 것**을 고른다
- effort는 계산량이다. 없는 지식이나 잘못된 근거는 고치지 못한다

C · 비용

요금은 100만 토큰당 달러, 입력과 출력이 따로다

$0.10
입력 100만 토큰
gpt-6-luna

$0.50
출력 100만 토큰
입력의 5배

$0.01
캐시에 맞은 입력
입력의 1/10

$10 · $50
GPT-6 Astra
입력 · 출력, luna의 100배

```
비용 = (입력 토큰 × 0.10 + 출력 토큰 × 0.50) ÷ 1,000,000
예) 입력 100,000 + 출력 10,000  →  (10,000 + 5,000) ÷ 1M = $0.015
```

핵심 출력은 입력의 1/10인데 비용은 1/3. 모델 이름 하나로 비용이 **100배** 달라진다 (단가는 2026-10 기준)

C · 비용

출력이 비싼 이유: 입력은 한꺼번에, 출력은 한 토큰씩 계산한다

입력 = Prefill

- 입력 1,000토큰을 **한 번의** 병렬 계산으로 처리한다
- GPU가 모델 가중치를 한 번 읽고 끝낸다

출력 = Decode

- 출력 1,000토큰이면 계산을 **1,000번** 반복한다
- 토큰마다 가중치를 다시 읽는다 → 메모리 속도가 병목

핵심 그래서 비용과 시간을 줄이는 첫 번째 방법은 **출력을 짧게 받는 것**이다

프롬프트 캐싱: 앞부분이 같으면 그 계산을 다시 쓴다

요청 1	**지시 + 문서 2,538토큰** 캐시 쓰기 · 단가 ×1.25	**질문 A** 일반
요청 2	**같은 지시 + 문서 2,538토큰** 캐시 읽기 · 단가 ×0.1	**질문 B** 일반

줄어드는 것

- 앞부분의 입력 비용 (90% 할인)
- 앞부분 처리 시간

그대로인 것

- 출력 비용과 생성 시간
- 답은 매번 새로 만든다 (답 캐시가 아니다)

핵심 같은 앞부분으로 10번 보내면 쓰기 1 + 읽기 9 = **2.15배** (캐시 없으면 10배)

캐시를 맞히려면: 고정된 것은 앞에, 바뀌는 것은 뒤에

맞는다



안 맞는다



조건과 확인 (gpt-6-luna)

- 앞부분이 1,024토큰 이상이고, 토큰 단위로 **완전히 같아야** 한다
- 마지막 사용 뒤 30분 이상 유지되고, 다시 쓰면 연장된다
- 확인: usage.input_tokens_details.cached_tokens (못 맞혀도 오류 없이 0)

C · 파라미터

자주 쓰는 파라미터 일곱 개

파라미터	하는 일	언제 바꾸나
model	어느 모델을 쓸지	품질 · 비용 · 속도의 균형
instructions	역할 · 규칙 (시스템 지시)	모든 요청에 공통인 지시
input	질문 · 문서 · 대화 기록	요청마다
reasoning.effort	추론의 양	단순 작업 none, 복잡한 판단 high
max_output_tokens	출력(추론 포함) 상한	비용 상한. 너무 작으면 잘린다
text.format	출력 형식 (구조화 출력)	코드가 받아 쓸 때
store	응답을 서버에 저장할지	개인정보가 있으면 False

주의 temperature · top_p는 **effort가 none일 때만** 받는다. 지원 여부는 모델마다 다르다

출력 상한에 걸리면 오류가 아니라 '미완성' 응답이 온다

```python
r = client.responses.create(
    model="gpt-6-luna", input=prompt,
    max_output_tokens=300)          # 추론 토큰도 이 안에 든다
if r.status == "incomplete":
    print(r.incomplete_details.reason)   # "max_output_tokens"
```

왜 위험한가

- HTTP는 200이라 오류로 안 잡힌다
- 추론만 하다 **빈 답**이 올 수 있다
- 그래도 쓴 토큰은 과금된다

어떻게 하나

- status를 항상 확인한다
- 추론 모델은 상한을 넉넉히 (25,000 이상 권장)
- 짧은 답은 상한이 아니라 지시 · 스키마로

구조화 출력: 응답 형식을 스키마로 고정한다

```python
from pydantic import BaseModel

class Allowance(BaseModel):
    grade: str          # 직급
    amount_krw: int     # 하루 금액
    article: str        # 근거 조항

r = client.responses.parse(
    model="gpt-6-luna", instructions=RULES,
    input="과장 일비는?", text_format=Allowance)
a = r.output_parsed          # Allowance 객체
print(a.amount_krw * 3)      # 바로 계산
```

왜 쓰나

- 코드가 받아 쓰려면 형식이 매번 같아야 한다
- 파싱 실패 · 필드 누락 · 타입 오류가 사라진다
- **형식만** 보장한다. 내용이 맞는지는 따로 검증한다

응답 객체: 답은 output 안에, 청구서는 usage에

```
{"id": "resp_…",
 "status": "completed",
 "model": "gpt-6-luna",
 "output": [
   {"type": "reasoning", …},
   {"type": "message",
    "content": [{"type": "output_text",
                 "text": "하루 30,000원"}]}],
 "usage": {"input_tokens": 328,
           "output_tokens": 52}}
```

읽는 법

- output은 목록이다. 추론 · 도구 호출 · 메시지가 섞인다
- **순서가 아니라 type으로** 찾는다. output〔0〕이 답이라는 보장은 없다
- SDK의 r.output_text가 메시지 글을 모아 준다
- status · usage는 항상 함께 기록한다

C · 로그

호출마다 로그 한 줄: 비용 · 지연 · 실패를 설명할 수 있게

```
{"ts": "2026-11-07T10:31:02", "model": "gpt-6-luna", "status": "completed",
 "input_tokens": 2598, "cached_tokens": 2538, "output_tokens": 200,
 "cost_usd": 0.000131, "latency_s": 1.2, "prompt_version": "v2"}
```

남기는 것

- 시각 · 모델 · status
- 토큰 수 (입력 · 캐시 · 출력)
- 비용 · 걸린 시간 · 프롬프트 버전

남기지 않는 것

- API 키
- 원문 · 개인정보 (이름 · 전화번호)
- 필요하면 원문 대신 ID · 해시만

핵심 로그가 없으면 '어제 얼마 썼나', '왜 느렸나'에 **답할 수 없다**

배운 것을 우리 프로젝트에 대 보기

매일 바뀐 공고 500~600건의 특징 문구를 같은 지시문으로 태깅한다고 하자

Q1 캐시가 맞으려면 지시문 · 예시 · 공고 내용을 어떤 순서로 놓아야 할까?

Q2 태그를 필터로 쓰려면 스키마에 어떤 필드와 값 목록이 필요할까?

Q3 태깅에 effort를 켤까, 끌까? 무엇으로 정할까?

Q4 화요일 리뷰에서 '어제 AI 비용'을 보여 주려면 로그에 무엇이 있어야 할까?

지금 답하지 않아도 된다. 화요일 리뷰에서 각자 담당에 가까운 질문 하나씩 같이 본다

실습 12 · 25분

실습 12: OpenAI Responses API

코랩에서 열기: labs/12_openai_api.ipynb

순서

- §1 가장 짧은 호출 · §2 usage로 비용 함수
- §3 instructions vs input
- §4 reasoning.effort 비교
- §5 비결정성 · §6 구조화 출력
- §7 스트리밍 · §8 오류 401 · 429
- §9 프롬프트 캐싱

보면서 확인할 것

- effort별 reasoning_tokens와 비용
- 같은 질문 세 번의 답 차이
- 두 번째 호출의 cached_tokens
- 끝나면 🤔 생각해보기 셀에 답을 적는다

정리

오늘 가져갈 열 문장

1. API는 주소 · 헤더 · 본문으로 요청하고, 상태 코드 · JSON으로 응답받는 약속이다
2. 4xx는 고쳐서, 429 · 5xx는 기다렸다가 다시 보낸다
3. 모델은 토큰을 읽고 쓴다. 비용 · 한도 · 속도가 모두 토큰 단위다
4. 한국어와 숫자는 잘게 쪼개진다. 계산은 코드에 맡긴다
5. 모델은 대화를 기억하지 않는다. 앱이 기록을 매번 다시 보낸다
6. LLM은 다음 토큰 예측을 반복한다. 그래서 환각과 비결정성이 생긴다
7. 추론 토큰은 안 보이지만 출력 단가로 과금된다
8. 출력은 입력보다 5배 비싸다. 짧게 받는 것이 가장 큰 절약이다
9. 고정된 앞부분은 캐시된다. 고정은 앞에, 바뀌는 것은 뒤에
10. 형식은 스키마로 고정하고, 호출마다 usage를 로그로 남긴다

정리

화요일(11/10) 리뷰까지

RE:봄 질문 하나에 답 적어 오기

- 블록 A · B · C의 녹색 장 질문 중 자기 담당과 가까운 것 하나
- 정답보다 **근거**를 본다: 계산이면 식, 설계면 이유
- 리뷰 때 한 사람 5분 안쪽

선택 과제

- 실습 12 §10 '바꿔서 실행' 셀 끝까지
- 오늘 실습을 구조동물 공고 데이터로 다시 해 보기 (노트북은 따로 공유)

생각 막히면 기다리지 말고 셀 번호와 오류 첫 줄을 바로 올린다. 키는 지우고

다음 시간

2부 (11/14): 모델 다루기와 임베딩

오픈 모델을 직접 띄운다

- 모델 카드 · 라이선스 · 메모리 계산
- 양자화로 작은 GPU에 올리기
- Ollama로 OpenAI 호환 서버 (실습 05)

프롬프트와 임베딩

- 프롬프트 설계 · 구조화 출력 심화 · 전처리
- 글과 이미지를 벡터로, 의미로 찾기
- 실습 09 · 10 (EmbeddingGemma 2)

실습 준비: 코랩에서 **T4 GPU 런타임**이 잡히는지 미리 확인해 둔다